

Detection Engineering & Purple Team Validation Report

Redshift SOC

Attack Emulation, Analytic Development & Detection Coverage Scoring

Engagement Scorecard

Metric	Result
Techniques executed	5 / 5 (T1046, T1595.003, T1110.001, T1027/T1059.001, T1218)
ATT&CK tactics spanned	Reconnaissance, Discovery, Credential Access, Execution, Defense Evasion
Detection analytics authored	5 (see Section 4)
Detection gaps identified	1 (Sysmon channel ACL)
Detection gaps remediated	1 / 1 (100%)
Post-remediation detection coverage	5 / 5 at Maturity Level 2+
Mean time to detect	< 60 seconds (log-to-index latency)

Executive Summary

This engagement simulated a realistic attack progression against the Redshift SOC environment. We executed five techniques spanning four ATT&CK tactics to evaluate whether our telemetry pipeline (Sysmon, Windows Event Log, Linux logs, pfSense syslog) and Splunk deployment could capture and generate detections at each stage.

All five techniques produced the expected telemetry. One issue emerged: local Windows process execution events initially failed to reach Splunk. We traced this to a Windows Event Log channel access-control list restriction and remediated it during the engagement.

Each technique now has a custom Splunk analytic, written to generalize beyond this exercise's test values. For example, brute-force detection uses failure-rate thresholds rather than matching an exact password list. All analytics are scored using a four-level detection maturity model (see Section 5).

Methodology

Framework

Technique selection follows MITRE ATT&CK (Enterprise). Each technique is documented with its ID, associated tactic, and the data sources the analytic depends on. This makes coverage claims traceable to specific, named telemetry rather than loose descriptions.

Environment & Constraints

Executed against the Redshift SOC lab. Attacker platform: **Kali Linux** on an isolated segment. Targets: **Windows 10 endpoint** and **Ubuntu/Apache server** on the LAN, reachable via **firewall-permitted(pfsense)** paths only (ICMP to Windows; TCP/22 and TCP/80 to Ubuntu).

Techniques 4 and 5 (local Windows execution) ran directly on the target. This reflects real-world scenarios where code execution occurs after an assumed prior compromise. The lab's firewall design (default-deny) doesn't permit direct delivery from the attacker segment to the Windows endpoint.

Scope Boundaries

No live malware, no unpatched CVE exploitation, no targets outside the lab's private subnets. All detection analytics were validated against live data from this engagement, not written speculatively.

Attack Narrative

The following reconstructs the engagement as a single incident timeline, showing what a defending analyst would observe and act on at each stage.

T+0:00 – Reconnaissance

External host 192.168.10.50 initiates a full-port TCP scan with service/version detection against 192.168.20.20. The firewall's default-deny posture limits visibility to two explicitly permitted ports (22, 80); all others return filtered—an observable signal of a hardened perimeter.

T+0:04 – Target Enumeration

The actor pivots to content discovery with a high-volume wordlist scan against the web root, searching for unlinked paths, backup files, admin interfaces, and debug endpoints.

T+0:11 – Credential Access

With SSH confirmed open, the actor attempts password-guessing against a known or likely username, consistent with harvested credentials or testing common defaults.

T+0:19 – Assumed Foothold

Modeling assumption: the actor has obtained execution on the Windows endpoint (e.g., via a phishing payload outside this engagement's scope). From here, the actor executes a base64-obfuscated PowerShell command—typical staging behavior.

T+0:23 – Defense Evasion

The actor invokes mshta.exe with inline script content—a signed-binary proxy execution technique to blend activity into normal process streams.

Detection Engineering

Each analytic below is scheduled-search-ready using generalized thresholds, so it remains valid against real, unknown future activity.

T1046 – Network Service Scanning

Tactic: Reconnaissance | Data Source: DS0029 Network Traffic

Detection Logic: A single source contacting six or more distinct destination ports on the same host within one minute is anomalous. The risk_score tiering feeds risk-based workflows rather than flat alerts.

- **Key SPL Elements:**

Match firewall syslog entries | Extract source, destination IPs and ports | Bin by 1-minute window | Count unique ports per source/target | Alert on >= 6 distinct ports

False Positives: Legitimate scanners (Nessus, OpenVAS) will trigger this. Recommend an allow-list of known scanner IPs. Load balancers performing health checks can produce lower-confidence hits; the tiering keeps these below high-confidence threshold.

```

mujtaba@kali: ~
Session Actions Edit View Help
zsh: corrupt history file /home/mujtaba/.zsh_history
(mujtaba@kali)~$ nmap -sV -Pn -p- 192.168.20.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-29 01:51 -0700
Nmap scan report for 192.168.20.20
Host is up (0.0029s latency).
Not shown: 65533 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.5 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.66 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 143.51 seconds
(mujtaba@kali)~$

```

Splunk output:

>	8/29/26 2:00:54.000 PM	Aug 29 14:00:54 192.168.20.1 Aug 28 17:14:12 filterlog[41900]: 1,6,,1000000103,em2,match,block,in,4,0 x0,,64,46196,0,DF,17,udp,79,192.168.10.50,192.168.189.2,34663,53,59 host = 192.168.20.1 source = udp:514 sourcetype = syslog
>	8/29/26 2:00:49.000 PM	Aug 29 14:00:49 192.168.20.1 Aug 28 17:14:07 filterlog[41900]: 1,6,,1000000103,em2,match,block,in,4,0 x0,,64,46195,0,DF,17,udp,79,192.168.10.50,192.168.189.2,34663,53,59 host = 192.168.20.1 source = udp:514 sourcetype = syslog
>	8/29/26 2:00:49.000 PM	Aug 29 14:00:49 192.168.20.1 Aug 28 17:14:07 filterlog[41900]: 1,6,,1000000103,em2,match,block,in,4,0 x0,,64,46194,0,DF,17,udp,79,192.168.10.50,192.168.189.2,34663,53,59 host = 192.168.20.1 source = udp:514 sourcetype = syslog
>	8/29/26 2:00:44.000 PM	Aug 29 14:00:44 192.168.20.1 Aug 28 17:14:02 filterlog[41900]: 1,6,,1000000103,em2,match,block,in,4,0 x0,,64,17513,0,DF,17,udp,67,192.168.10.50,192.168.189.2,45444,53,47 host = 192.168.20.1 source = udp:514 sourcetype = syslog
>	8/29/26	Aug 29 14:00:44 192.168.20.1 Aug 28 17:14:02 filterlog[41900]: 1,6,,1000000103,em2,match,block,in,4,0

T1595.003 – Wordlist Scanning

Tactic: Reconnaissance | Data Source: DS0015 Application Log, DS0029 Network Traffic

Detection Logic: Legitimate users don't generate dozens of unique-path 404s per minute from a single source. The 80%+ not-found ratio filters out normal browsing and accidental broken-link traffic, isolating automated wordlist enumeration.

• Key SPL Elements:

Parse Apache/IIS access logs | Count total requests and 404 responses | Bin by 1-minute window | Calculate not-found ratio | Alert on >= 20 requests with >= 80% 404 rate

False Positives: Broken web crawlers or misconfigured link checkers can mimic this pattern. The 20/minute threshold is tuned above typical accidental-404 volume.

```
Session Actions Edit View Help
$ gobuster dir -u http://192.168.20.20 -w /usr/share/wordlists/dirb/common.
txt
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
[+] Url: http://192.168.20.20
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
Starting gobuster in directory enumeration mode
.hta (Status: 403) [Size: 318]
.htaccess (Status: 403) [Size: 318]
.htpasswd (Status: 403) [Size: 318]
index.html (Status: 200) [Size: 143]
server-status (Status: 403) [Size: 318]
Progress: 4613 / 4613 (100.00%)
Finished
(mujtaba@kali)-[~]
```

Splunk Output:

status	count
200	2
403	4
404	4609

T1110.001 – Brute Force: Password Guessing

Tactic: Credential Access | Data Source: DS0028 Logon Session, DS0002 User Account

Detection Logic: Five or more failed authentications from one source within five minutes exceeds normal typo-rate. The unique_usernames_tried dimension separately flags username-spraying (many users, few attempts) versus classic brute force (one user, many passwords).

- Key SPL Elements:

Parse Linux auth logs | Extract failed login source IP and target username | Bin by 5-minute window | Count failed attempts and unique usernames | Alert on ≥ 5 failures

Opportunity: Feed a second-stage correlation search checking for a successful authentication from the same source shortly after. The sequence (failures, then success) is higher-confidence than either alone.

False Positives: A legitimate user with an expired/rotated password and cached credentials in an automated script (cron job, CI/CD pipeline) can trigger this. Cross-reference against a service-account allow-list before escalating.

Evidence: hydra terminal output

```
mujtaba@kali: ~  
$ echo -e 'password123\nadmin123\nSoclab123\nletmein\nqwerty123' > /tmp/small.txt  
$ hydra -i soclab -P /tmp/small.txt ssh://192.168.20.20  
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-29 02:11:37  
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4  
[DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (1:1/p:5), ~1 try per task  
[DATA] attacking ssh://192.168.20.20:22/  
1 of 1 target completed, 0 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-29 02:11:42  
$
```

and Splunk brute force analytic

Events Patterns <u>Statistics (1)</u> Visualization				
Show: 20 Per Page Format Preview: On				
src_ip	_time	failed_attempts	unique_usernames_tried	risk_score
192.168.10.50	2026-08-29 14:10:00	5	1	65

T1027 / T1059.001 – Obfuscated Command Execution

Tactic: Execution / Defense Evasion | Data Source: DS0009 Process (Process Creation)

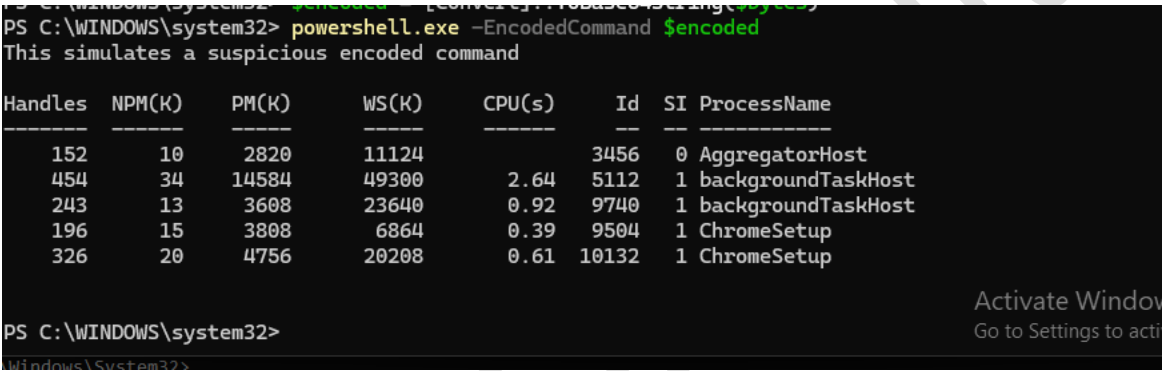
Detection Logic: Matches PowerShell's encoded-command flag (-e, -en, -enc, -encodedcommand) followed by base64 payload. Attackers often abbreviate flags to evade naive string matches.

• Key SPL Elements:

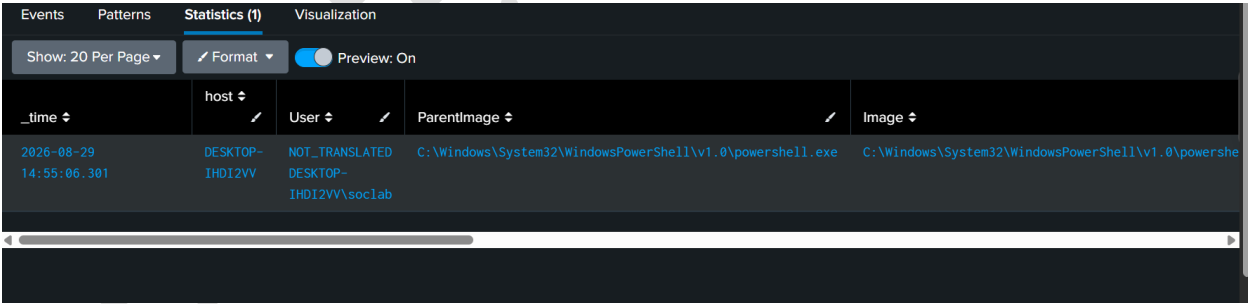
Parse Sysmon EventCode=1 (Process Creation) | Match CommandLine containing PowerShell encoding flags | Look for base64-pattern payload (20+ alphanumeric chars) | High risk_score (90)

Critical Note – Gap Identified & Remediated: This analytic initially returned zero results despite EventCode=1 being confirmed present in Windows Event Viewer. Root-cause: Windows Event Log channel ACL denied Splunk forwarder read access to Sysmon Operational channel. Fixed during engagement; analytic validated post-fix.

Evidence:



Splunk result post-remediation



T1218 – System Binary Proxy Execution (mshta.exe)

Tactic: Defense Evasion | Data Source: DS0009 Process (Process Creation, Process Lineage)

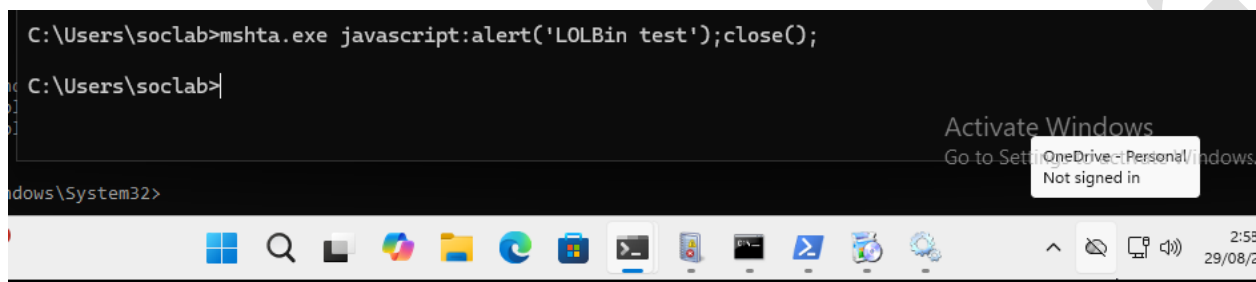
Detection Logic: mshta.exe execution alone isn't malicious (rare legitimate uses exist). This scores on two stackable signals: unexpected parent process + inline script content. Both together match real-world LOLBin abuse.

- **Key SPL Elements:**

Parse Sysmon EventCode=1 for mshta.exe | Check ParentImage against whitelist (explorer.exe, iexplore.exe) | Match CommandLine for inline script protocols (javascript:, vbscript:) | Score 95 if both unexpected signals present

False Positives: Legacy line-of-business applications occasionally shell out to mshta.exe. Recommendation: environment-specific allow-list keyed on full CommandLine.

Evidence: Splunk analytic results for mshta.exe execution



Splunk analytic

Show: 20 Per Page ▾ Format ▾ Preview: On							
_time ▴	host ▴	ParentImage ▴	Image ▴	CommandLine ▴	suspicious_parent ▴	script_inline ▴	risk_s
2026-08-29 14:57:30.915	DESKTOP-IHD12VV	C:\Windows\System32\cmd.exe	C:\Windows\System32\mshta.exe	mshta.exe javascript:alert('LOLBin test');close();	unexpected	yes	

Detection Coverage & Maturity Scoring

Each technique is scored against a four-level maturity model. This distinguishes 'we have the log' from 'we have a working, tuned alert'—a crucial difference in engineering effort and operational readiness.

Level	Name	Definition
0	No Visibility	Relevant log source is not collected.
1	Visibility Only	Log source is centralized and searchable, but no analytic exists—detection depends on manual search.
2	Analytic Exists	Working SPL analytic exists and is validated, but not yet scheduled or alerting.
3	Scheduled / Alerting	Analytic runs on schedule and generates alerts without manual action.

4	Actioned / Integrated	Alert is integrated into documented response workflow and triage runbooks.
---	-----------------------	--

Current State: All five techniques sit at Level 2 (validated analytic, manual execution)—the correct state for an interactive lab environment. Section 7 defines the roadmap to advance each to Level 3/4.

Root Cause Analysis – Sysmon Forwarding Gap

Symptom

EventCode=1 (Process Create) events confirmed present in local Sysmon Operational log but absent from Splunk, while Security and System channel events from the same host forwarded normally.

Initial Hypotheses Ruled Out

inputs.conf syntax (verified correct), forwarder service health (confirmed Running), forward-server connectivity (confirmed Active), Splunk disk/search availability (separately resolved as unrelated).

Root Cause

WinEventLogChannel::init reported errorCode=5 (Access Denied) specifically for the Microsoft-Windows-Sysmon/Operational channel. This channel carries stricter default ACLs than standard Windows channels (Security, System), and the forwarder's NT SERVICE virtual account was not included in the permissions.

Remediation

Explicit wevtutil channel-access grant naming required account types (SYSTEM, Administrators, Backup/Server Operators, Event Log Readers, Network Service), followed by forwarder service restart.

Verification

Absence of new errorCode=5 entries post-restart, followed by fresh test event (mshta.exe execution) confirmed indexed in Splunk within expected latency.

Why This Matters: The distinction between 'a log source is configured' and 'a log source is actually being collected' is exactly the kind of gap a production SOC discovers during an audit rather than during an incident. Finding and closing it here demonstrates the exercise's value.

Recommendations & Roadmap

1. Promote all five analytics to Maturity Level 3

Convert each to a Splunk scheduled search with defined cron cadence and notable-event action. Move from manual execution to automated triggering.

2. Implement T1110.001 correlation search

Make the failed-attempt spike followed by successful authentication the first Triarc-eligible analytic. Sequence is stronger signal than single events.

3. Add scheduled audit search for infrastructure health

Monitor Windows Event Log channel ACLs and Splunk forwarder health metrics to catch recurrence of the Sysmon gap before it impacts production.

4. Extend T1046 analytic with rule consistency check

Correlate detected open ports against the firewall's actual permitted-rule set. Automatically flag any scan result inconsistent with intended exposure.

5. Use these analytics as training data for Triarc

As Triarc comes online, use alerts from these five analytics plus deliberately generated benign look-alike activity as the labeled training set.

End of Report

Created by Mijtaba